

Universidad Cooperativa de Colombia
Ingeniería de Sistemas · 2026-II
8010 · Momento Evaluativo 1 (E1) — Proyecto Integrador
Isaac D. Vergara F.
10 de septiembre de 2026

01 · Resumen del proyecto

Nombre provisional: Núcleo de datos de Mtr Play.

Problema: Mtr Play vende accesorios tecnológicos y cuentas de streaming por WhatsApp y redes sociales, gestionando pedidos, clientes y catálogo de forma manual. Antes de construir cualquier aplicación, es necesario comprender qué reglas debe cumplir esa información para que un pedido no quede mal registrado, duplicado o sin cliente identificado.

Usuario: la persona que administra Mtr Play (Isaac), y en una siguiente etapa, los clientes que hacen pedidos por catálogo.

Propósito de la futura aplicación: registrar pedidos con uno o más productos por cliente, mantener un catálogo confiable, y responder preguntas sobre qué se vende, quién compra más y a dónde se despacha, evitando los errores de datos que este informe documenta.

02 · Dataset

Fuente: Online Retail Dataset, donado por el Dr. Daqing Chen al UCI Machine Learning Repository (<http://archive.ics.uci.edu/ml/datasets/Online+Retail>), obtenido a través de la copia pública del repositorio Spark: The Definitive Guide de Databricks (<https://github.com/databricks/Spark-The-Definitive-Guide>).

Qué representa: transacciones reales de una tienda en línea del Reino Unido entre 2010 y 2011: cada fila es una línea de un pedido, con el producto, la cantidad, el precio, el cliente y el país.

Por qué fue seleccionado: aunque Mtr Play vende tecnología y no artículos de decoración, la estructura del problema es la misma que enfrentará su aplicación — pedidos con una o varias líneas, clientes identificados, productos de catálogo y precios — y el volumen y las inconsistencias reales de este dataset permiten descubrir problemas de datos que Mtr Play aún no tiene registrados por sí mismo. Del archivo original (541.909 filas) se tomó un recorte real de diciembre de 2010 (42.481 filas) como archivo de partida, sin alterar ningún valor.

03 · Qué descubrimos

El perfilado del mes de diciembre de 2010 encontró los siguientes problemas, cada uno con una consecuencia concreta si la futura aplicación no lo controlara:

Hallazgo en los datos	Filas	Riesgo para la futura aplicación
Líneas sin CustomerID	15.631 de 42.481 (37%)	No se podría contactar al cliente para confirmar o despachar un pedido.
Filas exactamente duplicadas	500	Un pedido podría facturarse o descontarse del inventario dos veces.
Líneas con precio unitario = 0	273 (casi todas sin descripción)	Ventas reales quedarían registradas con valor \$0, distorsionando reportes.
Líneas con cantidad ≤ 0 fuera de cancelaciones	798	El inventario podría descontarse o sumarse mal si una devolución no queda distinguida de una venta.
Códigos de producto con más de una descripción	31	El mismo producto se mostraría con nombres distintos según el pedido.
Códigos que no son productos	16 códigos	Cargos y ajustes se mezclarían con productos reales

(POST, DOT, M, BANK CHARGES...)	distintos	en los reportes de ventas.
---------------------------------	-----------	----------------------------

04 · Cómo debería almacenarse

Se propone un modelo relacional de cuatro tablas, normalizado para no repetir datos de cliente o producto en cada línea de pedido:

Tabla	Columnas clave	Restricciones
clientes	customer_id (PK), pais	NOT NULL en pais
productos	stock_code (PK), descripcion	NOT NULL en descripcion
pedidos	invoice_no (PK), customer_id (FK), fecha, es_cancelacion	FK → clientes, NOT NULL, CHECK es_cancelacion IN (0,1)
lineas_pedido	id (PK), invoice_no (FK), stock_code (FK), cantidad, precio_unitario	FK → pedidos y productos, CHECK cantidad<>0, CHECK precio_unitario>0, UNIQUE(invoice_no, stock_code, cantidad, precio_unitario)

Los códigos que no son productos reales (POST, DOT, M, BANK CHARGES, etc.) se conservaron dentro de la tabla productos como "productos especiales" (envío, ajustes), en vez de dejarlos como texto libre, para que también queden protegidos por las mismas reglas de integridad que el resto del catálogo.

Con los datos válidos se cargaron 948 clientes, 2.801 productos, 1.708 pedidos y 26.351 líneas de pedido. Al intentar insertar las 500 filas duplicadas del archivo original, la restricción UNIQUE rechazó 496 de ellas automáticamente.

05 · Reglas para nuevos datos

Además de las restricciones ya incorporadas al esquema, se comprobó con una prueba directa que la base de datos rechaza información inválida sin depender de que el código de la aplicación la valide primero:

Dato inválido enviado	Regla que lo protege	Resultado real
Pedido con un customer_id que no existe en clientes	FOREIGN KEY pedidos → clientes	RECHAZADO: "FOREIGN KEY constraint failed"
Línea con precio_unitario = 0	CHECK (precio_unitario > 0)	RECHAZADO: "CHECK constraint failed: precio_unitario > 0"
Línea con un stock_code inexistente	FOREIGN KEY lineas_pedido → productos	RECHAZADO: "FOREIGN KEY constraint failed"

Estas reglas responden directamente a los hallazgos de la sección 03: exigir cliente y precio, e impedir productos y duplicados inválidos.

06 · Preguntas útiles

Se construyeron tres consultas SQL pensadas para decisiones reales de Mtr Play:

1. Productos más vendidos (para decidir qué mantener en catálogo o stock). En el mes analizado, el producto con más unidades vendidas alcanzó 5.139 unidades, seguido de piezas con 3.000–3.600 unidades — una diferencia útil para priorizar reabastecimiento.
2. Clientes con mayor valor total comprado (para atención prioritaria o fidelización). El cliente principal del mes acumuló £27.834,61 en compras, muy por encima del segundo (£19.950,66), lo que evidencia una alta concentración de valor en pocos clientes.
3. Pedidos y valor total por país (para decidir cobertura de envíos). El Reino Unido concentró 1.291 de los pedidos del mes y £496.477,34 del valor total, frente a Alemania como segundo mercado con solo 30 pedidos — una brecha que orientaría dónde enfocar logística primero.

07 · Rendimiento

Se probó la consulta "buscar todas las líneas de un producto específico" (necesaria para revisar el historial de ventas de un producto antes de reabastecerlo). Antes de crear un índice sobre `lineas_pedido.stock_code`, `EXPLAIN QUERY PLAN` mostró un `SCAN` completo de la tabla: SQLite debía revisar las 26.351 líneas cargadas para encontrar las de un solo producto.

Después de crear el índice `idx_lineas_stock_code`, el mismo `EXPLAIN QUERY PLAN` cambió a `SEARCH` `lineas_pedido USING INDEX idx_lineas_stock_code (stock_code=?)`: la búsqueda deja de recorrer toda la tabla y salta directamente a las filas del producto solicitado. Con 26 mil filas la diferencia en tiempo es pequeña, pero esta misma consulta dejaría de crecer al mismo ritmo que la tabla cuando Mtr Play acumule meses de pedidos reales.

08 · Hacia dónde seguirá el proyecto

En E2, este mismo núcleo de datos se conectará a un servicio real (por ejemplo, el backend del catálogo web de Mtr Play), reemplazando el dataset de reemplazo por los pedidos reales que la tienda empiece a generar, y se trabajará un pipeline reproducible para cargar esos datos con las mismas reglas de integridad ya definidas.

En E3, sobre la misma base, se incorporará la aplicación funcional: el flujo de "Confirmar pedido" descrito en la actividad de HTTP de Optativa II encaja directamente aquí, usando una transacción para registrar el pedido y sus líneas como una sola unidad — tal como se demostró en la sección 11 del notebook, donde un pedido con una línea inválida se revirtió por completo en vez de quedar guardado a medias.